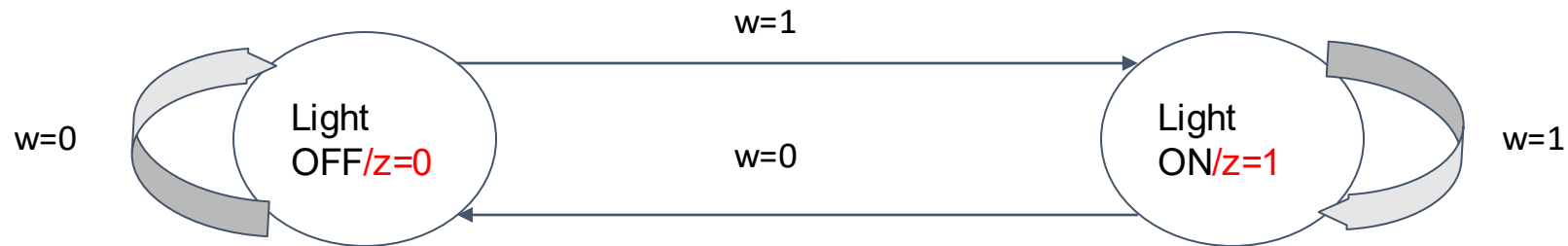


CSE460: VLSI Design

Lecture 8: Finite State Machines (Part 1)

A very simple controller[a light switch!!]

Light OFF $\rightarrow$ Light OFF [same state] [switch OFF, $w=0$]
Light OFF $\rightarrow$ Light ON [change of state] [switch ON, $w=1$]
Light ON $\rightarrow$ Light ON [same state] [Switch ON, $w=1$]
Light ON $\rightarrow$ Light OFF [change of state] [switch OFF, $w=0$]



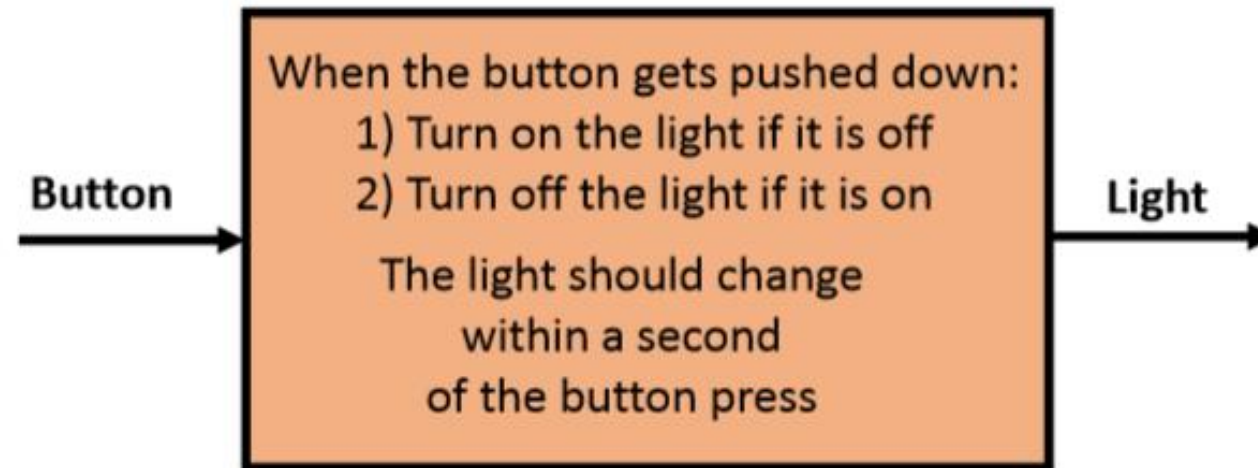
WHAT HAPPENS TO THE OUTPUT??

Light OFF $\rightarrow z=0$

Light ON $\rightarrow z=1$

Let's change the problem!!

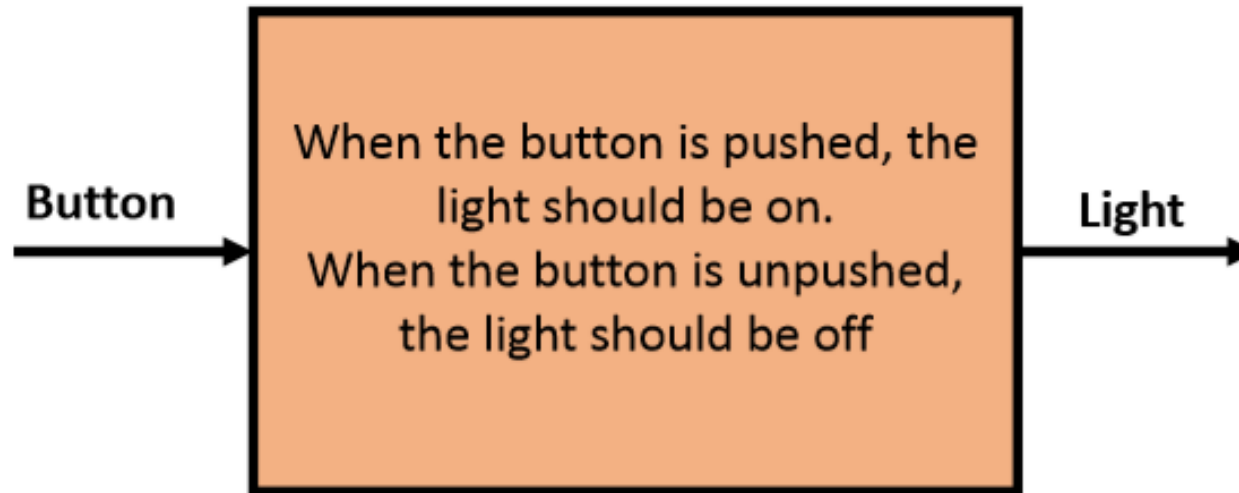
- What if you were given the following design specification:



What makes this circuit so different from those we've discussed before?

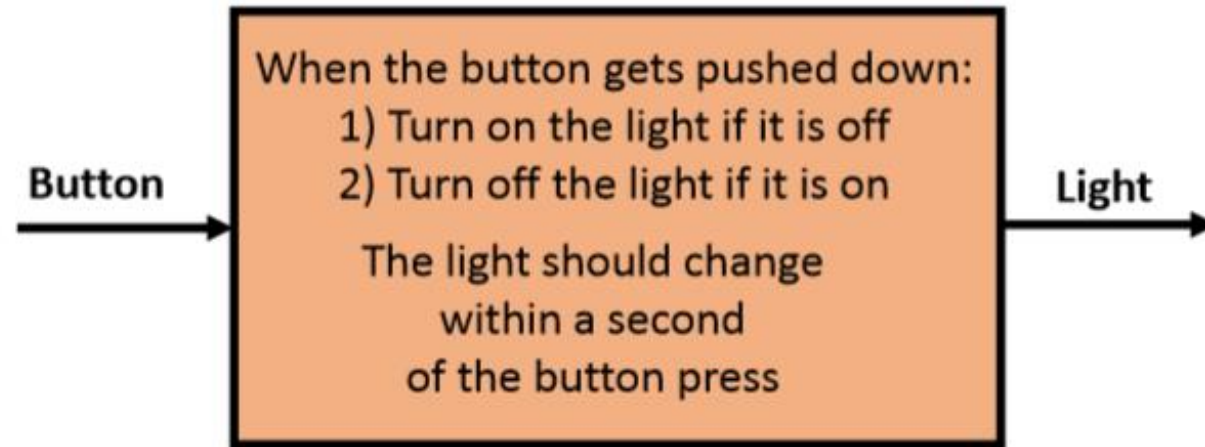
Something We Can Build (So Far)

- What if you were given the following design specification:



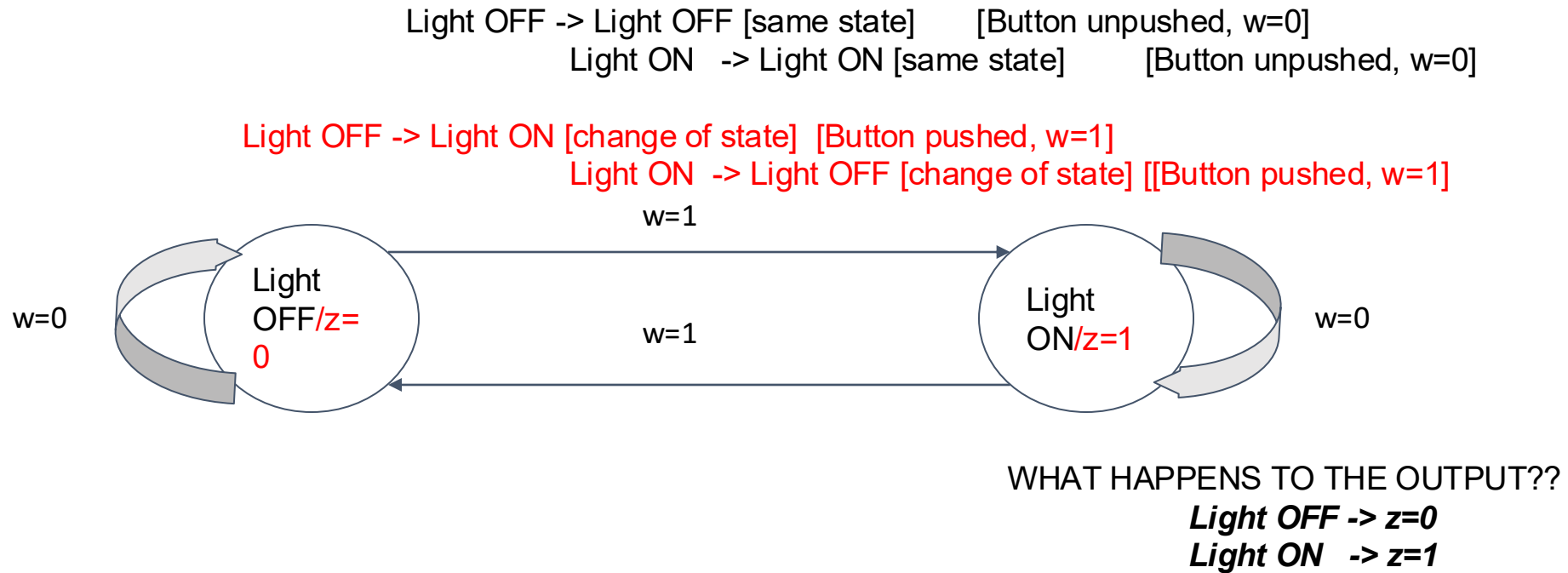
What makes this circuit so different from those we've discussed before?

Specifications:



1. “State” –i.e. **the circuit has memory** (become “state-ful”).
2. The output was changed by an input “event” (pushing a button) rather than an input “value” (output may depend on previous inputs).

Illustration of the previous example

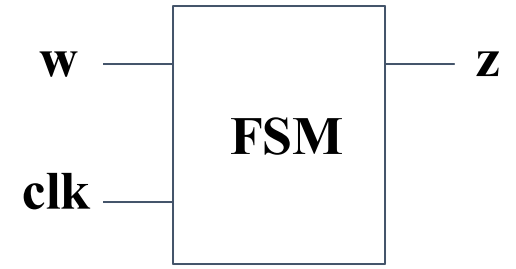


Example 1

An automatically-controlled vehicle is designed to run at some predetermined speed. However, due to some operational conditions the speed may exceed the desirable limit, in which case the vehicle has to be slowed down. To determine when such action is needed, the speed is measured at regular intervals.

- Let a binary signal w indicate whether the speed exceeds the required limit, such that $w = 0$ means that the speed is within acceptable range and $w = 1$ indicates excessive speed.
- The desired control strategy is that if $w = 1$ during two or more consecutive measurements, a control signal z must be 1 to cause the vehicle to slow down. Thus, $z = 0$ allows the current speed to be maintained, while $z = 1$ reduces the speed. Let a signal Clock define the required timing intervals, such that the speed is measured once during each clock cycle.

Example 1



From previous example 1, we wish to design the circuit that meets the following specification:

1. The circuit has one input **w**, and one output **z**.
2. All changes in the circuit occur on the **positive edge** of a clock signal.
3. The output **z** is equal to 1 if during two immediately preceding clock cycle the input **w** was equal to **1**. Otherwise the value of **z** is equal to **0**.

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0

Figure 3 : Sequences of input and output signals.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence)

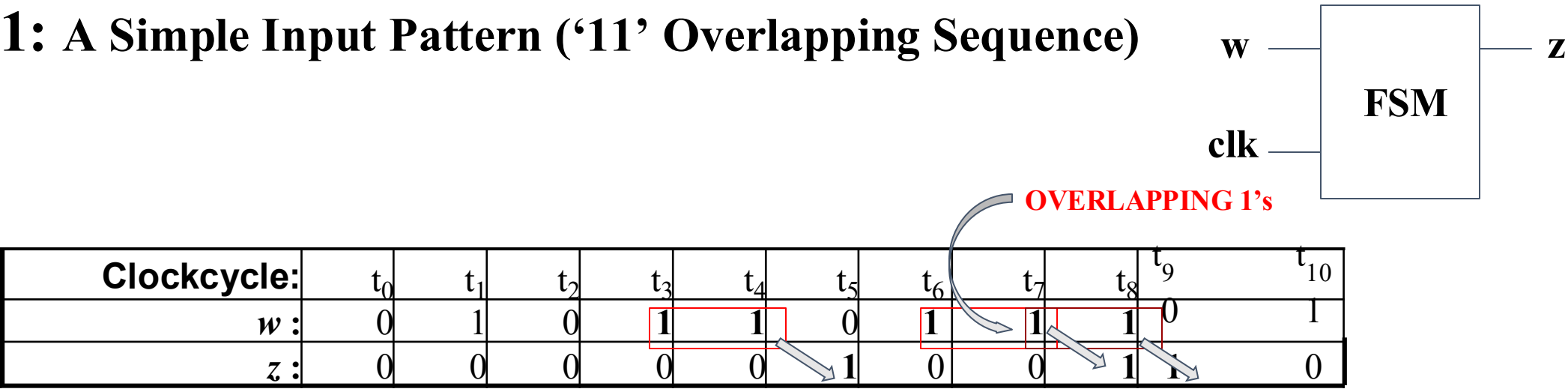
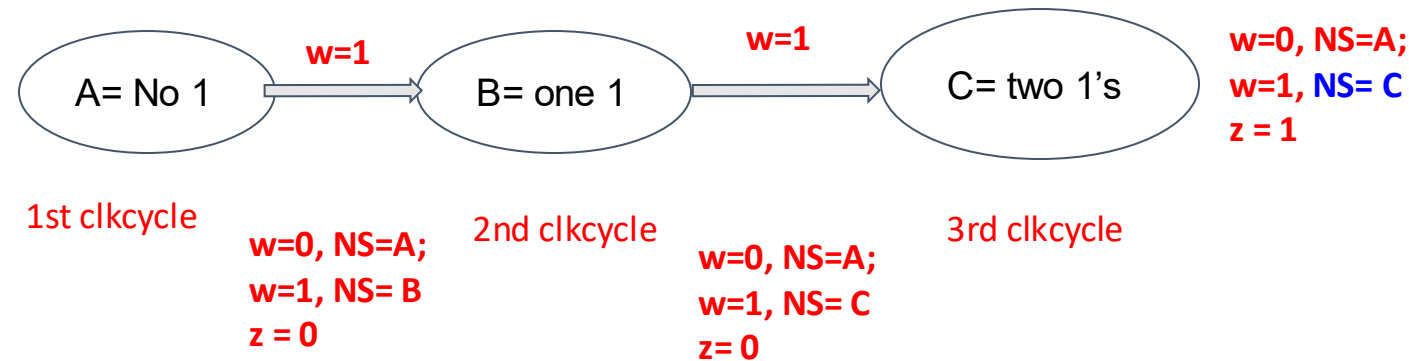


Figure 4 : Sequences of input and output signals.

The circuit detects two consecutive '1'. So, it must keep in memory how many '1' it received in its input. So, a state diagram is introduced.

- **A** state- no '1' in input,
- **B** state- one '1' in input and
- **C** state- two '1' in input.



Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit - Moore Type

State Diagram

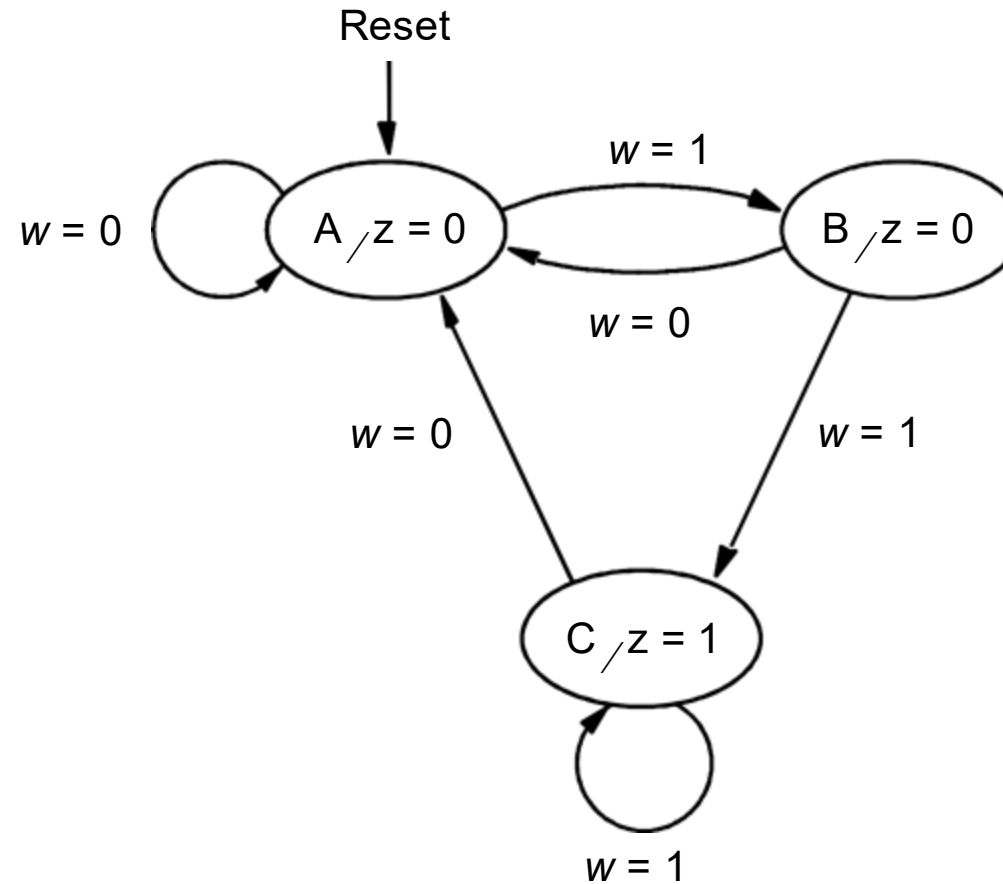


Figure 5: State diagram of a simple sequential circuit.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [State Table]

State
Table

Present state	Next state					Output z
	$w = 0$		$w = 1$			
A	A		B			0
B	A		C			0
C	A		C			1

Note:
n bits can
represent
 2^n states.

Figure 6: State table for the sequential circuit in Figure 5.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [State Assigned Table]

	Present state	Next state		Output z
		$w=0$	$w=1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

We need to find expressions (K-map) for

- next state (Y_2, Y_1)
- output (z).

Note that,

$$\triangleright Y_1 = f(w, y_1, y_2)$$

$$\triangleright Y_2 = f(w, y_1, y_2)$$

$$\triangleright z = f(y_1, y_2)$$

Figure 7: State-assigned table for the sequential circuit in Figure 7.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [K-Map]

		y_2y_1			
w		00	01	11	10
	0	0	0	d	0
	1	1	0	d	0

		y_2y_1			
w		00	01	11	10
	0	0	0	d	0
	1	0	1	d	1

y_2	y_1	
	0	1
0	0	0
1	1	d

Ignoring don't cares

$$Y_1 = wy_1y_2$$

$$Y_2 = wy_1y_2 + wy_1y_2$$

$$z = y_1y_2$$

Using don't cares

$$Y_1 = wy_1y_2$$

$$Y_2 = wy_1 + wy_2$$

$$= w(y_1 + y_2)$$

$$z = y_2$$

Figure 8: Derivation of logic expressions for the sequential circuit in Figure 7.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [Circuit]

Note that,

- # flipflops = # bits to represent current state (#y)
- Outputs of flipflops correspond to current state bits (y_1, y_2)
- Inputs of flipflops correspond to next state bits (Y_1, Y_2)

In this problem, we need **two flipflops** as we have used **two bits** (y_1, y_2) to assign all the present states (A, B, C).

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [Circuit]

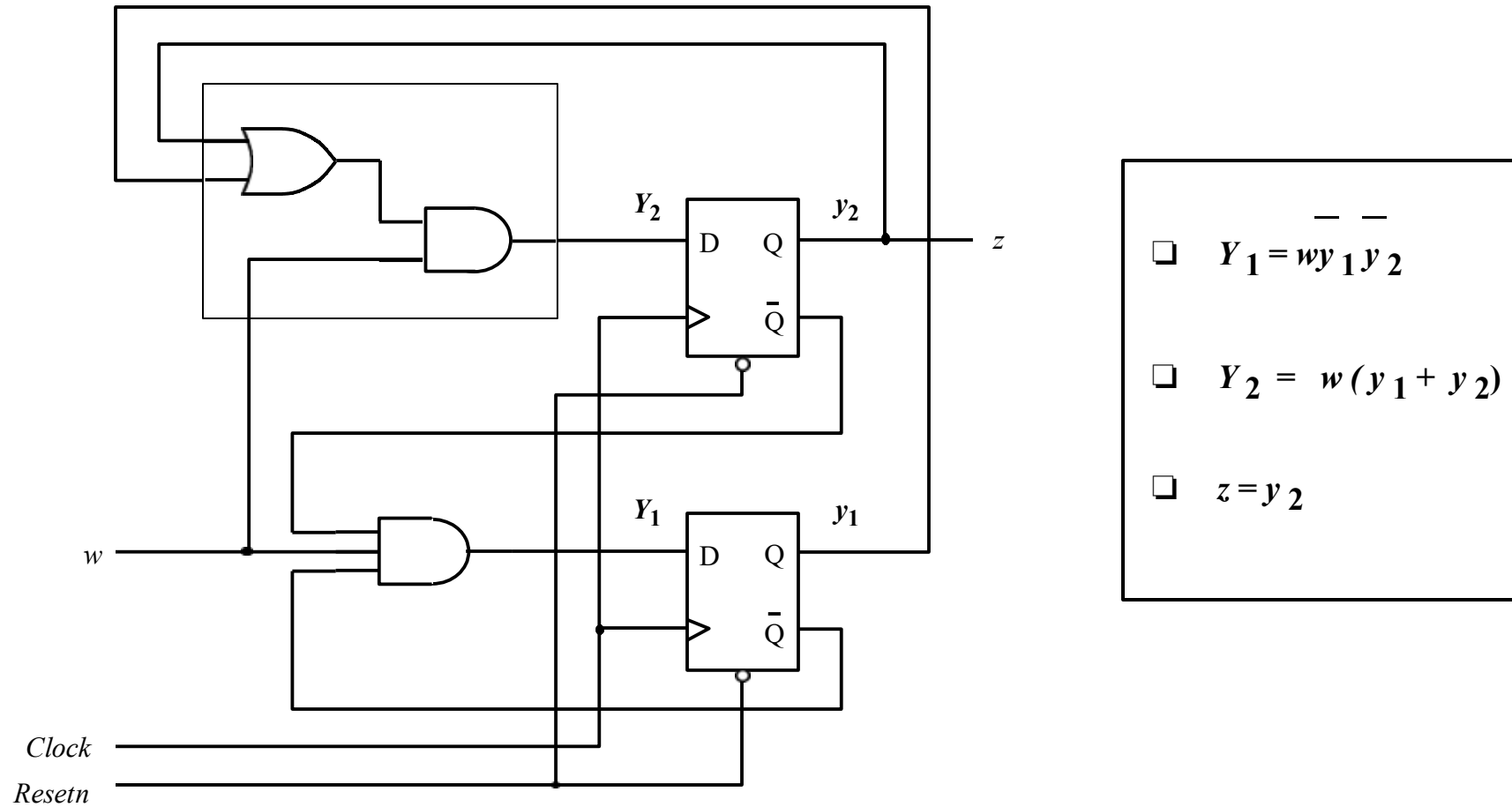


Figure 9. Final implementation of the sequential circuit derived in Figure 8.

Q4. [CO2] - Marks 10

You are tasked with designing a CD/DVD drive's control circuitry using a Moore-type finite-state machine. The drive has one input push button w , which generates $w = 1$ when pushed, and $w = 0$ when un-pushed. A $w = 1$ input causes the tray to open if it was already closed, or causes the tray to close if it was already open. The control circuit has only one output signal, z , which determines if the CD/DVD drive should be open ($z = 1$) or closed ($z = 0$) with the following instructions:

- If the tray is closed, pushing the button (setting $w = 1$) should generate the signal ($z = 1$) to open the tray
- If the tray has been open for less than 3 clock cycles, pushing the button (setting $w = 1$) should generate the signal ($z = 0$) to close the tray
- The tray should automatically close after being open for 3 clock cycles
- For simplicity, assume that the time required for opening and closing the tray is negligible compared to the system clock's time period

Q6. [CO1] - Marks 10

A car was in a stand still condition. To *start* the car, a start/stop button, w should be pressed. An output, $z = 1$ confirms that the car is moving (accelerating) or running (optimum constant speed).

- If $w = 0$, the car should remain still initially. For, $w = 1$, the car should start moving (if it was still) or stop (if it has just started moving, but not running yet).
- Once moving, the car should reach the optimal speed within two consecutive clock cycles without pressing the start/stop button, w .
- After two cycles, it will automatically gain the optimal speed to keep running. The car should maintain its speed independent of the value of w while running.
- The break, b can only be pressed for an emergency stop while running. If the car is still or moving, break, b can be ignored.

[Hint: The system has two inputs: w , b and one output: z]

Summary

Steps->

- State diagram
- State table
- State assigned table
- K-map (next states & outputs)
- Circuit

Example 1(cont.): A Simple Input Pattern ('11' Non-Overlapping Sequence) (Detection Circuit - Moore Type)

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	0	0

NON-OVERLAPPING 1's

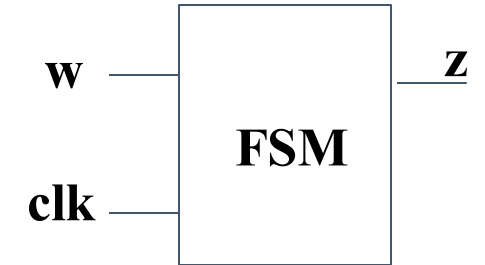
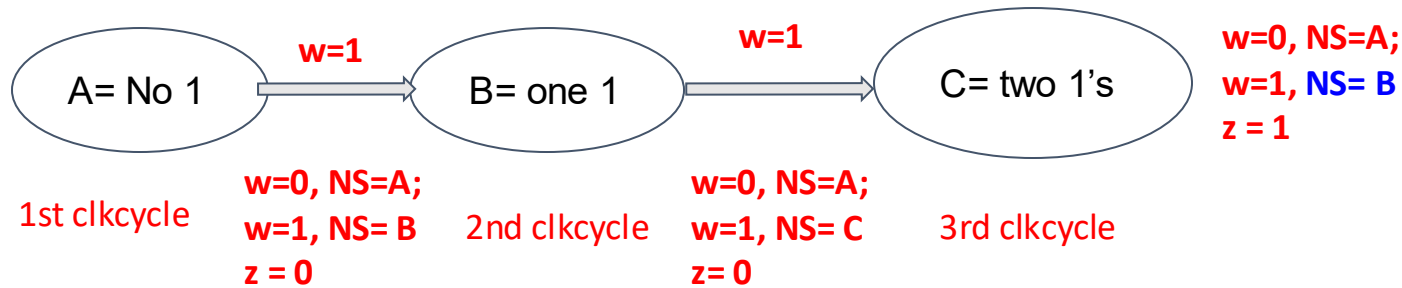
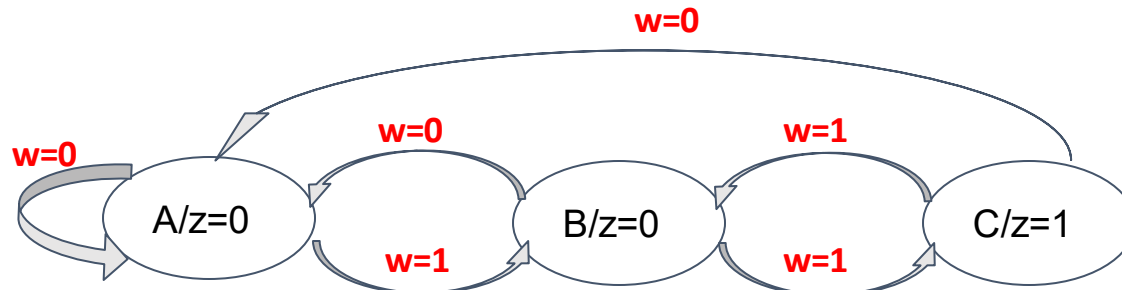


Figure 4 : Sequences of input and output signals.



STATE DIAGRAM



General form of a sequential circuit

- W represents input/s
- Z represents output/s
- Flip-flops are used to save the “present” state of the circuit, Q
- Also called Finite State Machine (FSM) or simply, machine
- 2 types: depending on the connection drawn in pink on the circuit

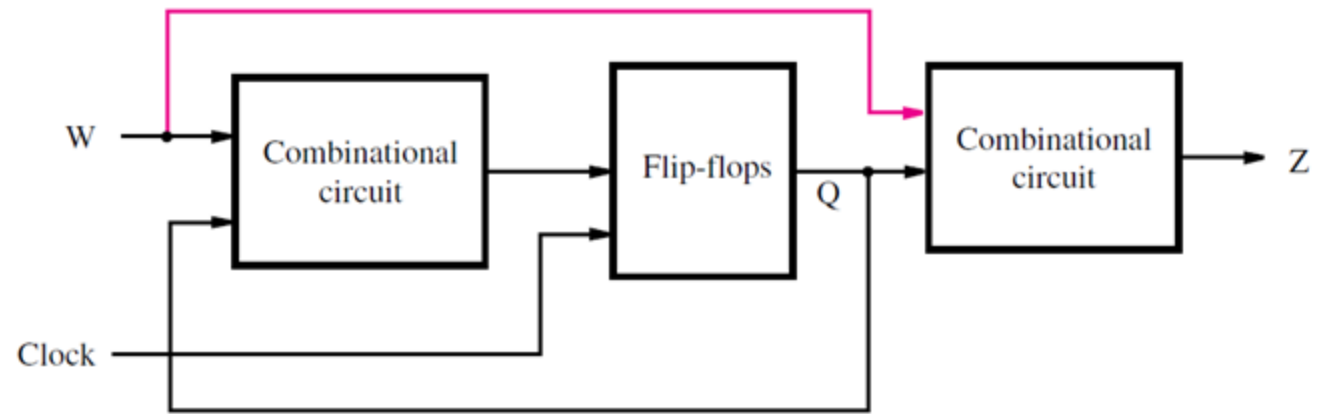
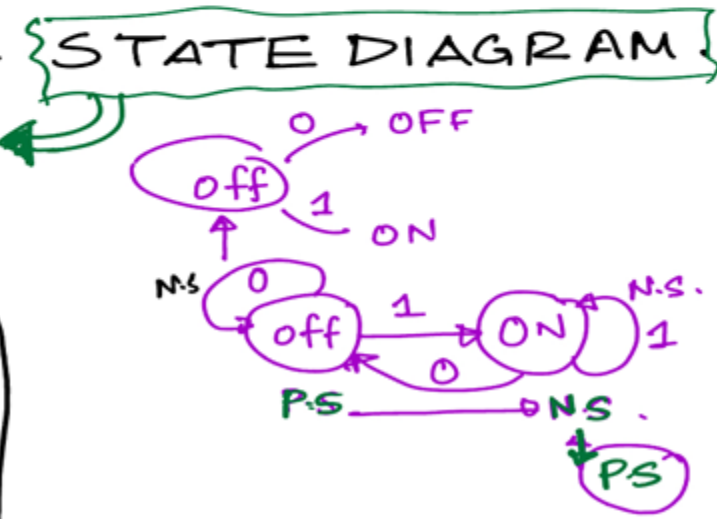
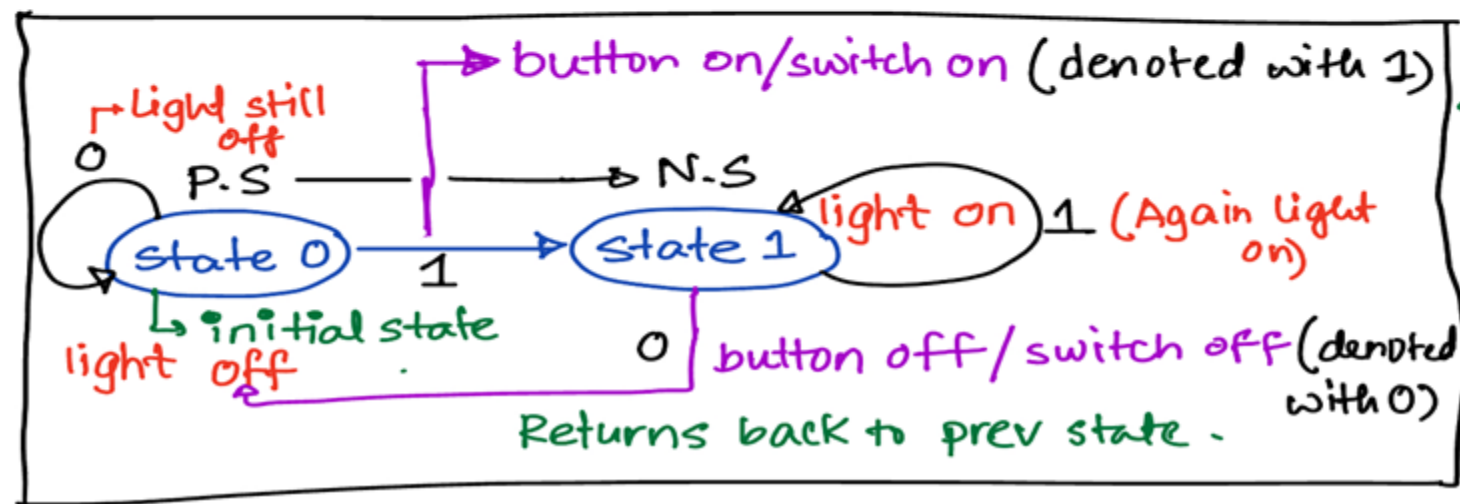
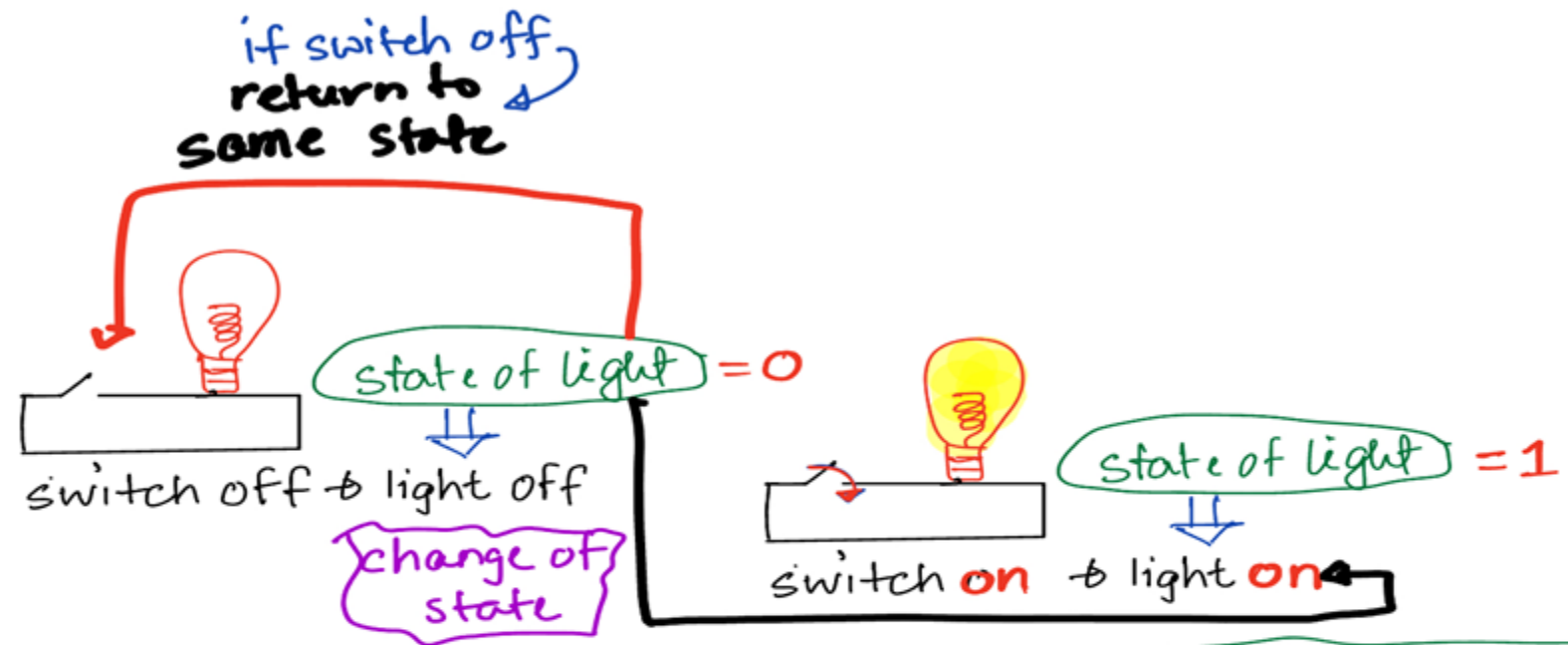


Figure: General form of a sequential circuit

Illustration of light switch problem

Illustration of previous example

Example: 1 Detector



Example: 1 Detector

State Table

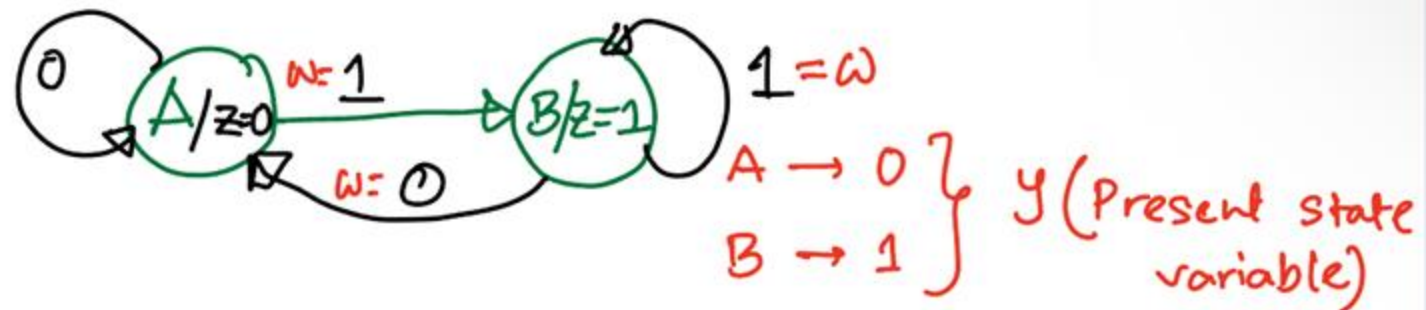
State 0 $\rightarrow$ A
State 1 $\rightarrow$ B

P.S	N.S		Output z
	w=0	w=1	
A	A	B	0
B	A	B	1

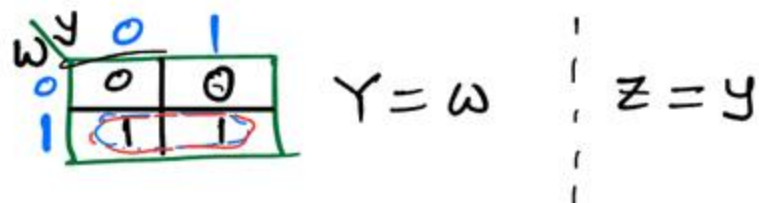
State Assigned Table

P.S _y	N.S (Y)		Output z
	w=0	w=1	
0(A)	0	1	0
1(B)	0	1	1

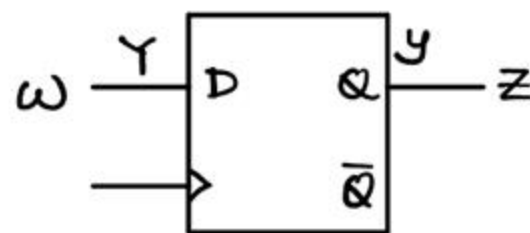
Representation of state Diagram



Find next state variable, Y & output, z



1 Detection Circuit



FSM types

- Moore type FSM
Output (Z) depends only on the “present” state (Q) of the circuit

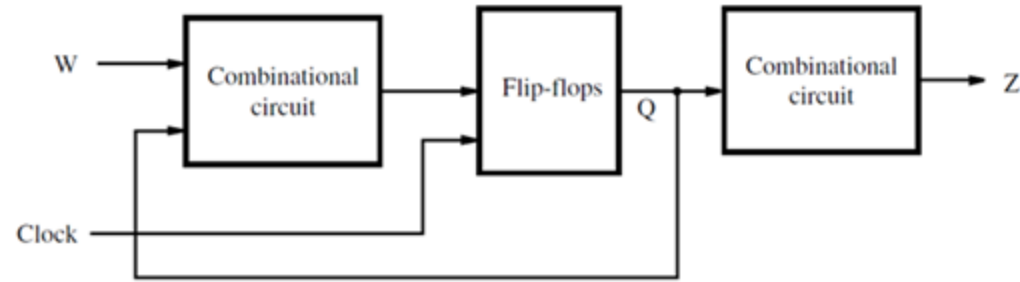


Figure: Moore type FSM

- Mealy type FSM
Output (Z) depends on both the primary inputs (W) and the “present” state (Q) of the circuit

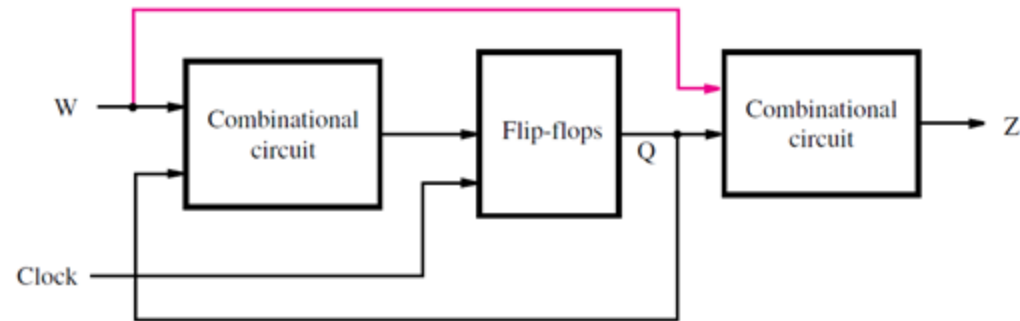


Figure: Mealy type FSM

Moore vs. Mealy type machines

Moore Machine	Mealy Machine
▪ Output depends only on the <i>present state</i>.	▪ Output depends both on the <i>present state</i> and the <i>present input</i>.
▪ Generally, it has <i>more states</i> than Mealy Machine.	▪ Generally, it has <i>fewer states</i> than Moore Machine.
▪ The output is a function of the current state and changes <i>at clock edges</i>	▪ The output is a function of the transitions and can change any instant
▪ In Moore machines, more logic is required to decode the outputs resulting in more circuit delays. They generally react one clock cycle later.	▪ Mealy machines react faster to inputs. They generally react in the same clock cycle.

4. Design a Mealy machine that simulates detecting fraudulent transactions in a stream of credit card transactions based on input categories:

- A: Low-value transaction.
- B: Medium-value transaction.
- C: High-value transaction.

The machine outputs 1 (fraud detected) if two consecutive high-value transactions (C) are followed by a low-value transaction (A). Otherwise, it outputs 0.

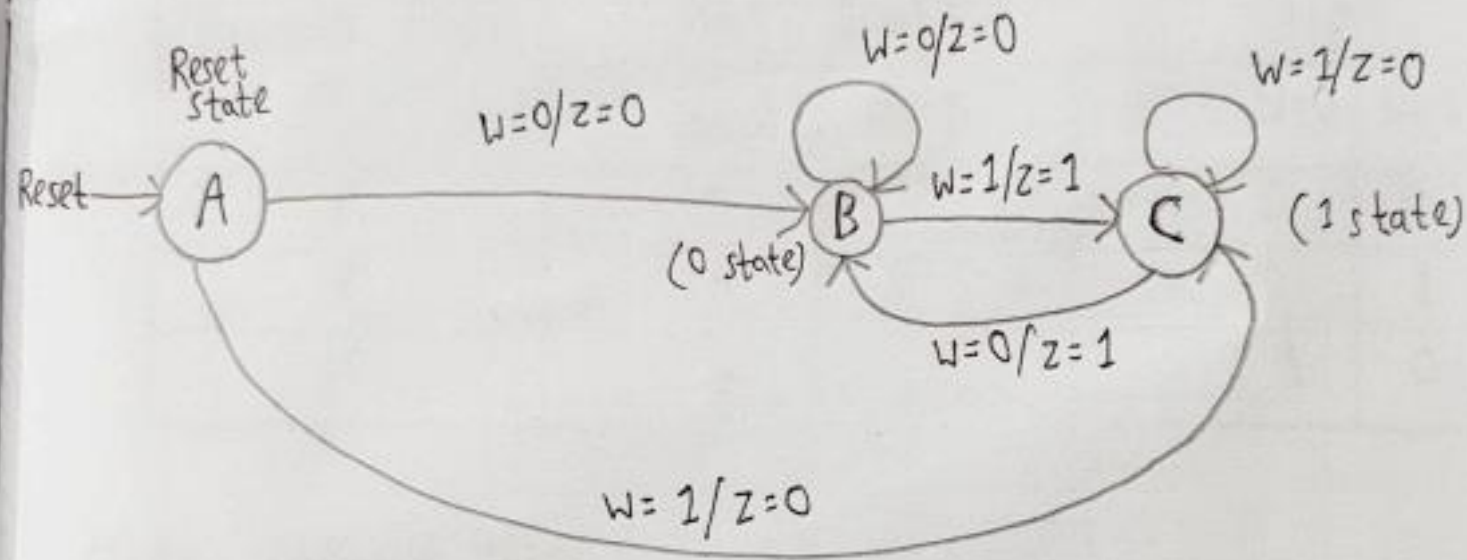
Example:

- **Input:** B C C A B
- **Output:** 00010

	(a)	Design the state diagram. Mark your transitions clearly.	3
	(b)	Derive the assigned table and write the Verilog code to implement the following FSM.	4
	(c)	Validate your answer with the appropriate input and output waveforms on a timing diagram. Include all possible combinations of the input (w).	3

(3)

(a)



This FSM here is of ~~no~~ mealy type. Here, the output(z) is dependent on both the current state and the current input.

(b)

State Table:

Present State	Next State		Output	
	W=0	W=1	W=0	W=1
A	B	C	0	0
B	B	C	0	1
C	B	C	1	0

State Assigned Table:

Present State	Next State		Output	
	w=0	w=1	w=0	w=1
00	01	10	0	0
01	01	10	0	1
10	01	10	1	0